

AMANE — Mission C

Infrastructure & résilience du cluster distribué

orchestrator-go · consensus (etcd · Patroni) · réplication CRDT · mesh WireGuard · serveur gRPC

1. Vue d'ensemble

Mission C porte l'infrastructure et la résilience du cluster AMANE : consensus distribué (etcd Raft), haute disponibilité PostgreSQL (Patroni, réplication synchrone), réplication CRDT delta multi-site, réseau intra-site WireGuard et serveur gRPC exposant le framework aux Missions A et B.

Jalons

Jalon	Contenu	État
1	Contrat proto B↔C, squelette gRPC (Ping, TLS, interceptors)	Terminé
2	Consensus : membership, quorum, lease/fencing etcd, Enroll/NotifyRevocation	Terminé
3	Réplication CRDT delta multi-site + Write/Read	Terminé
4	Mesh WireGuard intra-site + failover < 5 s, zéro perte	Terminé

2. Contrat proto B ↔ C

Source : proto/amane/framework/v1/framework.proto. Ne jamais renuméroter un champ ni changer son type (compatibilité binaire, contrôlée par buf lint et buf breaking en CI).

Services

RPC	Requête	Réponse	Description
	<code>1 PingRequest</code>	<code>1 PingResponse</code>	Interface 1 A ↔ C, ping (jalon 1).
	<code>2 EnrollRequest</code>	<code>2 EnrollResponse</code>	Interface 1 A ↔ C, membership.
	<code>3 WriteRequest</code>	<code>3 WriteResponse</code>	Interface 3 B ↔ C, Write.
	<code>4 ReadRequest</code>	<code>4 ReadResponse</code>	Interface 3 B ↔ C, Read.
	<code>5 NotifyRevocationRequest</code>	<code>5 NotifyRevocationResponse</code>	Interface 3 B ↔ C, NotifyRevocation.
	<code>6 PushDeltaRequest</code>	<code>6 PushDeltaResponse</code>	Interface 3 B ↔ C, PushDelta.

Messages

PingRequest

Champ	Type	
		■

PingResponse

Champ	Type	
<code>1 status</code>	<code>1 string</code>	■
<code>2 notification</code>	<code>2 string</code>	■
<code>3 version</code>	<code>3 string</code>	■
<code>4 server_time</code>	<code>4 google.protobuf.Timestamp</code>	■

EnrollRequest

Champ	Type	
<code>1 machine_id</code>	<code>1 string</code>	■
<code>2 auth_key</code>	<code>2 bytes</code>	■
<code>3 site_id</code>	<code>3 string</code>	■
<code>4 operator_id</code>	<code>4 string</code>	■

EnrollResponse

Champ	Type	■
message_id	string	1
cluster_id	string	2
enroll_at	google.protobuf.Timestamp	3

WriteRequest

Champ	Type	■
journal_id	string	1
op_id	uint64	2
encrypted_payload	bytes	3
site_id	string	4
month_id	string	5

WriteResponse

Champ	Type	■
conflict_id	uint64	1
notification	string	2
synced	bool	3
conflict_at	google.protobuf.Timestamp	4

ReadRequest

Champ	Type	■
journal_id	string	1
from_timestamp	uint64	2
limit	uint32	3

ReadResponse

Champ	Type	■
entries	repeated JournalEntry	1

JournalEntry

Champ	Type	■
seq_id	uint64	1
encrypted_payload	bytes	2
conflict_at	google.protobuf.Timestamp	3
site_id	string	4

NotifyRevocationRequest

Champ	Type	■
month_id	string	1
revoked_at	string	2
revoked_at	google.protobuf.Timestamp	3

Champ	Type	■
<code>reason</code>	<code>string</code>	<code>4</code>

NotifyRevocationResponse

Champ	Type	■
<code>quorum</code>	<code>bool</code>	<code>1</code>
<code>notification</code>	<code>string</code>	<code>2</code>

Delta

Champ	Type	■
<code>notification</code>	<code>string</code>	<code>1</code>
<code>index</code>	<code>int64</code>	<code>2</code>
<code>delta</code>	<code>int64</code>	<code>3</code>
<code>setpoint</code>	<code>uint64</code>	<code>4</code>

PushDeltaRequest

Champ	Type	■
<code>siteid</code>	<code>string</code>	<code>1</code>
<code>fromnode</code>	<code>string</code>	<code>2</code>
<code>delta</code>	<code>repeated Delta</code>	<code>3</code>

PushDeltaResponse

Champ	Type	■
<code>acked_seq</code>	<code>uint64</code>	<code>1</code>
<code>value</code>	<code>int64</code>	<code>2</code>
<code>notification</code>	<code>string</code>	<code>3</code>

Verrouillage par les tests (module `tests/`)

Module Go isolé `tests/` (package `tests/contracts`, `go.mod` github.com/amane/tests). `proto_contract_test.go` verrouille le contrat $B \leftrightarrow C$ par les numéros de champs exacts de tous les messages partagés et la présence des 6 RPC — équivalent programme de buf breaking sans dépôt git (tourne toujours). `contracts_test.go` valide l'Interface 1 $A \leftrightarrow C$ contre etcd réel (Enroll → registre, double-enroll → AlreadyExists, NotifyRevocation → quorum recalculé, révocation inconnue → NotFound) et le chemin d'écriture $B \leftrightarrow C$ gated : Write (leader) → synced + Read retourne le payload + PushDelta site-b → site-a converge (value -2, acked_seq 1) ; Write non-leader → codes.FailedPrecondition, zéro entrée au journal. Lancement : `cd tests && AMANE_TEST_ETCD=localhost:2379 go test ./... -count=1 -race` (etcd réel requis, skip sinon).

3. Consensus (etcd + Patroni)

Membership & quorum

Le registre des membres vit dans etcd (clés `/members/{machine_id}`). Un membre est créé par transaction conditionnelle (`createRevision == 0`) pour garantir l'unicité. Le quorum suit la majorité Raft : `quorum = total/2 + 1`. Une révocation recalcule le quorum et le retourne via NotifyRevocation.

Lease & fencing

Le lead orchestrateur : TTL 30 s, renouvellement périodique, élection via `concurrency.Session + Election.Campaign`. Fencing : un nœud qui perd sa lease ne peut plus écrire ; reprise du leadership journalisée en

JSON (log/slog) avec timing.

Fencing applicatif — Write gated par lease

`grpcserver.Server.Write` est refusé tant que le nœud ne détient pas la lease `etcd` (interface `grpcserver.Leadership`, câblée en prod via `consensus.NewLeadership/WithLeadership`) : `codes.FailedPrecondition` « write refusé : nœud non leader (fencing lease) » avant tout accès au journal (aucun compromis de séquence) — anti split-brain, l'élection `etcd` ne produit jamais deux leaders en même temps. Preuve live : 2 instances, Write sur le non-leader → `FailedPrecondition` puis commité (`node_id`, `synced=true`) après libération de la lease ; intégration `TestLeadershipFencingAgainstEtcd` (`AMANE_TEST_ETCD`). Relectures (`Read`, `Ping`, `Enroll`, `NotifyRevocation`) non gated ; sans `withLeadership` le gating est désactivé (tests/rétro-compat), jamais en prod.

Composant	Détail
quorum.go	ComputeQuorum(total) = majorité stricte
membership.go	Registry Add/Remove/Has/Quorum (txn etcd)
lease.go	Leadership : Session TTL 30 s, Election, fencing
client.go	NewClient(endpoints) client etcd v3

HA mesurée (protocole reproductible)

Scripts : `scripts/failover_measure.sh` `crash|partition` et `scripts/switchover_measure.sh`. La mesure est instrumentée (horodatage `T0`, watch `etcd` horodatée, sondage REST 50 ms) et vérifie le fencing (ancien primary = réplica, `pg_is_in_recovery=t`) et le zéro perte (marqueur acké côté standby sync présent sur le nouveau primary).

Scénario	failover_ms	writable_ms	cible	fencing	zéro perte
Switchover contrôlé (API Patroni)	≈ 2 200	≈ 2 400	< 5 s ATTEINTE	OK	OK
Crash (docker kill primary) — avec superviseur actif	≈ 3 200	≈ 4 000	< 5 s ATTEINTE	OK	OK
Partition réseau (docker network disconnect) — avec superviseur actif	≈ 3 400	≈ 3 400	< 5 s ATTEINTE	OK	OK

Lecture honnête : le plancher de lease est une validation **Patroni** (`patroni/config.py`, "can't be smaller than 20, adjusting") appliquée au paramètre `ttl`, indépendamment du backend DCS — **changer de DCS (ZooKeeper, Kubernetes) ne lève pas le plancher**. Sans superviseur, la détection d'un crash non contrôlé est donc bornée à ≈ 21 s ; avec le superviseur actif, la libération du lock `/service/<scope>/leader` avant expiration fait passer le crash sous la cible < 5 s (mesuré ci-dessus) ; le chemin **contrôlé** (switchover planifié) reste déjà sous 5 s via le release volontaire de la lease. Le choix architectural entre les 3 options est comparé ci-dessous.

Décision d'architecture HA — 3 options pour un crash < 5 s

Le lock Patroni est anti split-brain : personne ne capture le lock avant expiration de la lease de l'ancien primary. Les options B et C cherchent à réduire CE délai ; une 4e piste (changer de DCS) est écartée — prouvé ci-dessus, le plancher `ttl` est porté par Patroni, pas par `etcd/zk/k8s`.

Critère	A – Chemin contrôlé assumé	B – Baisser le plancher ttl	C – Orchestrateur externe
Détection crash (SIGKILL)	≈ 2 s (lease)	≈ 3-5 s (ttl court)	≈ 0,5-1 s (sondes actives)
Bascule totale (crash)	≈ 21 s	≈ 4-6 s	≈ 1-3 s
Switchover planifié	≈ 2,2 s (prouvé)	≈ 2 s	≈ 2 s
Risque split-brain / faux failover	Nul (plancher intact)	Élevé (à-coups réseau ⇒ faux failover, split-brain)	Élevé (à-coups réseau ⇒ faux failover, split-brain)
Garantie zéro perte	OK (répl. synchrone)	OK mais période risquée	OK (répl. synchrone)
Fencing	OK (Patroni)	OK mais fenêtre fragilisée	OK (démotion + kill)
Effort Mission C	Aucun (déjà prouvé)	Moyen (patch <code>config.py</code> / override, retesté, stress réseau)	Élevé (composant, sa propre HA)
Dépendances	Aucune	Fork/maintenance Patroni	Composant superviseur + REST Patroni

Critère	A – Chemin contrôlé assumé	B – Baisser le plancher ttl	C – Orchestrateur externe
Cible < 5 s crash	NON (assumée ~20 s)	Étroitement, fragile	OUI si composant HA
Choix retenu (v1)			A – chemin con

Superviseur maison — option C retenue (décision d'architecture)

Le crash non contrôlé passe par un **superviseur dans orchestrator-go** (package supervisor/ : config.go, evaluator.go, beacon.go, patroni.go, supervisor.go), déployé sur les 3 nœuds. **Pourquoi ce choix** : (1) mono-repo Go, aucune nouvelle infrastructure (pas de Kubernetes, pas de Pacemaker) ; (2) etcd déjà présent — heartbeat **propre** du superviseur à lease courte (2 s), **indépendante** du plancher ttl >= 20 s de Patroni ; (3) pas de fork de Patroni, chemin de promotion déjà éprouvé à ~2,2 s en switchover ; (4) le superviseur **ne décide pas la promotion** : Patroni reste l'autorité de fencing et de promotion (réplique synchrone et zéro perte préservées) — simple déclencheur, pas une couche HA superposée ; (5) HA du superviseur : élection via un lock etcd « droit de forcer » (une seule instance agit), jamais de SPOF. **Mécanisme final** : le superviseur (1) publie dans etcd un **heartbeat propre à lease courte (2 s)** rapportant le rôle local ; (2) détecte le crash — probe REST du primary en échec **et** heartbeat etcd stale/absent après StaleConfirm=2 ticks — avec garde anti-partition : (heartbeat frais ⇒ primary vivant mais isolé ⇒ pas de forçage — split-brain partiel) ; en **coupure complète** (partition réseau), le heartbeat stoppe ⇒ forçage légitime (confirmé par la mesure : 3,0 s) ; (3) **libère le lock Patroni** en supprimant /service/<scope>/leader dans etcd (fencing conditionné : autorisé seulement quand le heartbeat du nœud est confirmé mort — l'ancien primary ne peut plus se battre) ; (4) appelle POST /failover **sans le champ** leader — avec leader, Patroni le traite comme switchover et attend un ancien joignable — et répond quand la promotion est finie (Patroni promeut en ~2-3 s une fois le lock libéré). force=true **seul ne suffisait pas** : sans libération du lock, la promotion restait bornée par l'expiration de la lease (ttl >= 20 s). Timing par défaut : ProbeInterval 500 ms, HeartbeatTTL 2 s, StaleConfirm 2, LockTTL 3 s, CoolDown 30 s.

Mesuré (stack réelle) : crash 3,2 s failover / 4,0 s writable ; partition (coupure complète, docker network disconnect) 3,0 s failover / 3,4 s writable — fencing + zéro perte OK sur les deux chemins.

Élément	Design
Détection	probe REST Patroni (500 ms) + heartbeat etcd propre (lease 2 s) ; crash si probe en échec ET heartbeat stale/absent
Déclenchement	libération lock etcd /service/<scope>/leader + POST /failover sans leader — Patroni répond 200 à la fin de promotion
Fencing	conditionné au heartbeat stale (nœud confirmé mort) — l'ancien primary ne recapture pas le lock ; Patroni reste l'autorité
Autorisation	lock etcd « droit de forcer » : une seule instance agit (élection ~ms)
Cas exclus	partition réseau / primary vivant : heartbeat frais ⇒ pas de forçage — Patroni tranche ; force=true seul insuffisant (sinon)

4. Réplication CRDT (delta multi-site)

Jalon 3 — terminé. Delta-CRDT / PN-Counter pour les quantités de stock : replication/counter.go (Apply, Merge max-par-nœud, Value), convergence déterministe quel que soit l'ordre des deltas (tests 3 sites, 0 conflit non résolu). Journal multi-site replication/journal.go : écriture append-only, séquences monotones, rejeu idempotent (même site_id + op_seq → même seq committé), pagination (limit ≤ 1000). Write/Read branchés sur le gRPC (Interface 3 B ↔ C) ; les payloads restent chiffrés (encrypted_payload de Mission A, jamais de clair côté C). **Jalon 4 — relay multi-site implémenté** (voir ci-dessous).

Relay multi-site — replication/relay.go (**jalon 4, garantie AP**). Un delta porte le **total cumulé** du nœud émetteur (Inc/Dec depuis son origine, pas un incrément) → fusion **max par nœud émetteur** : commutative, associative, idempotente — doublons, réordonnancements et trous de séquence ne compromettent jamais la convergence. Add met à jour total + pending + seq ; Accept fusionne côté récepteur (réflexion d'un delta auto-émis ignorée) ; Outgoing/Confirm(ackSeq) font le gc du pending (fuite de mémoire évitée). RPC **additif** PushDelta au contrat B ↔ C (numéros de champs stables ; code généré committé, buf generate OK) ; handler grpcserver via WithRelay(*replication.Relay) — sans relay → codes.Unavailable ; câblé au démarrage dans cmd/orchestrator/main.go. **Preuve live (grpcurl)** : site-eshop inc 5 puis 7 → ack seq 2, value 7 ; puis site-atelier dec 3 → value 4. Tests : relay_test.go (convergence 3 nœuds réordonnés/doublés, ack gc du pending, réflexion ignorée, trou de seq idempotent, merge associatif) + TestPushDeltaAcrossSites/TestPushDeltaWithoutRelay

(bufconn, 2 sites) — tout vert, y compris -race.

5. Mesh WireGuard (intra-site)

Jalon 4 — implémenté. Package mesh/ (orchestrator-go) : génération de la config wg0.conf par nœud — IP virtuelle stable par machine (10.10.<site>.<n>/24, inchangée au failover), AllowedIPs = /32 du pair (jamais de routage global 0.0.0.0/0 rejeté par validation), PersistentKeepalive = 25 (NAT/CGNAT), fichier .conf écrit en 0600 (clé privée jamais loggée). Usage strictement intra-site (etcd, Patroni, WAL) — l'inter-site passe par TLS sortant, jamais par WireGuard.

6. Serveur gRPC

Aspect	Choix
Transport	gRPC/TLS 1.3 minimum (MinVersion=tls.VersionTLS13), mTLS si -tls-ca fourni (RequireAndVerifyClientCertIfNotPresent)
Chargement credentials	server+client (client_credentials.go), CA interne, jamais InsecureSkipVerify
Erreurs	status.Error(codes.X, ...) — jamais d'erreur brute
Intercepteurs	Logging (slog JSON) + Recovery
Introspection	grpc reflection (grpcurl, dev)
Lancement	.env non requis — go run ./cmd/orchestrator

mTLS prouvé e2e (TCP réel, PKI générée en mémoire) : client valide OK, refus sans cert, refus cert d'une autre CA, refus d'un handshake TLS 1.2. Vérif manuelle : grpcurl -cacert/-cert/-key + openssl s_client -tls1_2 refusé.

7. Environnement local (docker-compose)

etcd (Raft, source de vérité Patroni) + deux nœuds Spilo/PostgreSQL 17 en réplication synchrone.
synchronous_commit=on, max_slot_wal_keep_size=1024MB.

Service	Image	Ports(hôte)
---------	-------	-------------

Vérification Patroni : curl -s localhost:8008/patroni (primary/replica, sync_state).

8. Commandes de développement

Commande	Effet
go build ./... / go test ./... -count=1	Build & tests (depuis orchestrator-go/)
docker compose up -d / down -v	Stack etcd + Patroni (up / reset volumes)
buf generate / lint / breaking	Génération & vérifs proto (racine)
~/go/bin/grpcurl -plaintext -d '{}' localhost:50051 .../Ping	Test manuel gRPC
~/go/bin/grpcurl -cacert tls/ca.crt -cert tls/client.crt -key tls/client.key -d '{}' 127.0.0.1:50051 .../mTLS/Ping	Test manuel mTLS (g)
go test ./grpcserver/ -run TestCredentials -v	Tests e2e TLS 1.3 / mTLS (PKI en mémoire)
~/go/bin/grpcurl -plaintext -d '{"journal_id":"stock",...}' .../Write	Écriture (Interface 3, rejeu idempotent)
AMANE_TEST_ETCD=localhost:2379 go test ./consensus/ -run MembershipIntegration	Intégration membership vs etcd réel
AMANE_TEST_ETCD=localhost:2379 go test ./consensus/ -run TestLeaderElection	Élection fencing lease-based (Write gated) vs etcd réel
cd tests && AMANE_TEST_ETCD=localhost:2379 go test ./... -count=1 -race	Tests cross-mission : contrats B↔C / A↔C + chemin d'écriture (etcd)
bash scripts/failover_measure.sh crash partition	Mesure failover (fencing + zéro perte)
bash scripts/switchover_measure.sh	Mesure switchover contrôlé (< 5 s)

Commande	Effet
<code>.venv/bin/python docs/generate_mission_c_pdf.py</code>	Régénère ce PDF (ReportLab)

9. Règles transverses (non négociables)

- Jamais de clé en clair : AK privée, DEK, KEK interdits en Go, logs, env ou messages.
- Réplication synchrone : `synchronous_commit=on`, `max_slot_wal_keep_size` borné, failover < 5 s, zéro perte.
- Logs structurés log/slog (JSON), corrélation de timing failover/lease.
- gRPC : `status.Error(codes.X, ...)` — jamais d'erreur brute, ni nil, nil.
- Contrat proto : pas de renumérotation/changement de type ; code généré engagé ; buf breaking en CI.